React JS

Learning ReactJS

PRAVEEN NAIR

What is React?

ReactJS is a popular JavaScript library

It is used for building user interfaces for web applications.

It was developed and maintained by Facebook

It is great for SPA which load once and later everything happens thru JavaScript without reloading the page.

React was release on 29th May 2013.

Similar products available in market are Angular and Veujs.

Installing React

Download and Install Node Js

node –version

npm – version (node package manager)

if nodem error try **npm install -g npm**

npx create-vite@latest myapp
npm create vite@latest myapp //better

Select react and then javascript

npm install, code . and then npm run dev

Vscode extention vscode es7 react/reduc/react-native/js snippets

React JSX (JavaScriptXML)

JSX allows us to write HTML elements in JavaScript and place them in the DOM

```
export default function App() {
  return (<h1>Hello World</h1>);
}
```

.....

Tags must be closed

One top Element / Fragment

```
<div>
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</div>


<>
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</>
```

React Components (jsx)

```
export default function Header() {
  return <h1>Company Name</h1>;
}

export default function Content() {
  return <h1>Display content here</h1>;
}

export default function Footer() {
  return <h1>Footer Section</h1>;
}
```

Component Props & Expression

....User.js.....

```
export default function User(props) {
  return (
    <div>Hello {props.name}</div>
  )
}
```

.... App.js

```
<div>
<User name="John"/>
    </div>
```

React Styling – Inline Styling

```
const Header = () => {
  return (
    <>
      <h1 style={{backgroundColor: “blue"}}>Hello World!</h1>
      <p>Add a little style!</p>
    </>
  );
}
```

.....

camelCase property name – used backgroundColor instead of background-color

React Styling – Importing directly

```
import ‘./Styles.css’;
const Student = () => {
  return <h1 className=“txtblue”>Hello Student!</h1>;
}
export default Student;
.........................................
Style.css
.txtblue {
  color: Blue;
  padding: 20px;
  font-family: Sans-Serif;
  text-align: center;
}
```

Events in React

```
function Customer() {
  const greet = () => {
    alert("Hello!");
  }
  return (
    <button onClick={greet}>Greet</button>
  );
}
```

Events (Passing Arguments)

```
function Customer() {
  const greet = (a) => {
    alert(a);
  }
  return (
    <button onClick={() => greet("Hi Ajay")}>Greet</button>
  );
}
```

State Management (useState hook)

The data is stored in the React Component's state object.

When the state object changes, the component will re-render itself.

```
export default function App6() {
  const [count, setCount] = useState(1); //cannot be inside loops, if or functions
  console.log(Date())  //runs everytime count changes
  return (
    <>
      <button onClick={() => setCount((prevState) => prevState - 1)}>-</button>
      <span>{count}</span>
      <button onClick={() => setCount((prevState) => prevState + 1)}>+</button>
   </>
  );
}
```

Spread Operator Arrays & Objects (...)

```
let score = [1,4,5];
 score = [...score, 9];
 console.log(score);
```

.....

```
let student = { name: "John",subject:"Maths" };
 student = {...student, marks:90};
 console.log(student);
```

React Router

npm i -D react-router-dom

- <Router> //Browserrouter as router

```
  <p><Link to="/add">Add</Link> |
        <Link to="/view">View</Link></p>
  <Routes>
    <Route index element={<Add />} />
    <Route path="add" element={<Add />} />
    <Route path="view" element={<View />} />
  </Routes>
</ Router >
```

useNavigate (react-router-dom)

```
import React from 'react’
import { useNavigate } from 'react-router-dom’
export default function Post() {
  const Navigate = useNavigate()
  const goToFeeds = () => {
    Navigate('/’)
  }
  return (
    <div>Post
      <button onClick={goToFeeds}>Go to Feeds</button>
    </div>

  )
}
```

Side Effects (useEffect hook)

```
export default function App6() {
  const [runs, setRuns] = useState(0);
  const [wickets, setWickets] = useState(0);
  //   console.log("Component loaded");
  useEffect(() => {
    if (wickets > 0) console.log("Better Luck Next Time!");
  }, [wickets]);
  useEffect(() => {
    if (runs > 0) console.log("Good Job!");
  }, [runs]);
  return (
    <>
      <button onClick={() => setRuns((prevState) => prevState + 1)}>
        Runs({runs})
      </button>
      <button onClick={() => setWickets((prevState) => prevState + 1)}>
        Wickets({wickets})
      </button>
    </>
  );
}
```

Components Life Cycle

Mounting, unmounting and updating
comment strictmode to prevent rerender twice

```
useEffect(() => {
    console.log("Component: Mounting");
    return () => {
      console.log("Component: Unmounting");
    };
  },[cnt,result]);
.......................................................
function Result(props) {
  const r = props.r;
  if (r) {
    return <Pass/>;
  }
  return <Fail/>;
}
```

useContext(child can access var)

```
import { useState, createContext, useContext } from "react“
import Child from "./Child";
export const UserContext = createContext();
export default function App13() {
const [user, setUser] = useState("John");
  const [email, setEmail] = useState("john@gmail.com")
  return (
    <>
    <UserContext.Provider value={{user, email}}>
      <h2>Hello {user} from App13 component</h2>
      <Child />
    </UserContext.Provider>
    </>
  );
}
..........................
import {UserContext} from "./App13“
import { useContext } from "react";
export default function Child() {
  const {user, email} = useContext(UserContext);
  return (
    <>
      <h2>Hello {user} from Child component</h2>
    </>
  );
}
```

Map function to display array

```
function Cart() {
  const arr = ["Rice", "Wheat", "Sugar"];
  return (
      <ul>
        {arr.map((item) => (
          <li>{item}</li>
        ))}
      </ul>
  );
}
```

Adding Env Variables

Create .env file in the root folder of the project

Every variable you define should start with VITE_ in .env file

Example : VITE_API_URL=http://localhost:8080

Now you can use the variable in any of your components as shown below

const API_URL = import.meta.env.VITE_API_URL

Restart the local server by running npm start

CRUD API (Axios)

```
const fetchUsers = async () => {
  const response = await axios.get(url);
  setUsers(response.data);
};

const updateUser = async () => {
  const data = {
    name: name,
    email: email,
  };
  await axios.put(url + userId, data);
};

const deleteUser = async () => {
  await axios.delete(url + userId);
};

const addUser = async () => {
  const data = {
    name: name,
    email: email,
  };
  await axios.post(url, data);
};
```

Ecomm shopping cart

Add to cart

Increase and decrease quantity

List and Keys

Keys are used by React to track individual items in a list, enabling it to manage changes like reordering, addition, or removal of elements. Without keys, React may not update the list correctly, leading to inefficient re-renders and potential UI bugs.

When rendering a list in React, each child in the list needs to have a unique key. Keys help React keep track of which list items correspond to which DOM elements. If the list is reordered or new items are inserted, React uses the keys to figure out which items stayed the same and which ones changed.

React Class vs Func Components

```
class Bus extends React.Component {
  render() {
    return <h2>This is a bus.</h2>;
  }
}
.....................................................

function Bus() {  //function based component is popular
  return <h2>This is a bus.</h2>;
}
.......................................................
root.render(<Bus />);
```

useRef

```
export default function Register() {
  const firstName = useRef();
  const handleSubmit = () => {
    const user = {
      firstName: firstName.current.value,
    };
    console.log(user);
  };
  return (
    <div className="App-Register-Row">
      <div style={{ backgroundColor: "white" }}>
        <h2>Registration Form</h2>
        <p>
          <input type="text" placeholder="Enter First Name" ref={firstName} />
        </p>
        <p>
          <button onClick={handleSubmit}>Submit</button>
        </p>
      </div>
    </div>
  );
}
```

Ternary Operator

```
.............................................
function Result(props) {
  const r = props.r;
  return (
    <>
      { r ? <Pass/> : <Fail/> }
    </>
  );
}
```

If Statement with props

```
function Result(props) {
  const r = props.r;
  if (r) {
    return <Pass/>;
  }
  return <Fail/>;
}
```

React-icons

```
npm install react-icons
.........................................................
import React from 'react’
import { FaHome, FaBookmark } from "react-icons/fa";
export default function App14() {
  return (
    <div>
        <p><FaHome/>Home</p>
        <p><FaBookmark />Feeds</p>
        <p>Post</p>
        <p>Album</p>
    </div>
  )
}
```

React Redux

npm install @reduxjs/toolkit react-redux

userSlice.js

```
import { createSlice } from "@reduxjs/toolkit";
const userSlice = createSlice({
  name: "user",
  initialState: { name: "", isLoggedIn: false },
  reducers: {
    login: (state, action) => {
      state.name = action.payload;
      state.isLoggedIn = true;
    },
    logout: (state) => {
      state.name = "";
      state.isLoggedIn = false;
    }
  }
});
export const {login,logout} = userSlice.actions
export default userSlice.reducer
```

store.js

```
import { configureStore } from "@reduxjs/toolkit";

import userReducer from "./userSlice.js";

export const store = configureStore({

  reducer: {

    user: userReducer,

  },

});
```

main.js

```
import User from "./User.jsx";
import { store } from "./store.js";
import { Provider } from "react-redux";
createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <User />
  </Provider>,
);
```

User.jsx

```
import { useSelector, useDispatch } from "react-redux";

import { login, logout } from "./userSlice";

export default function User() {

  const user = useSelector((state) => state.user);

  const Dispatch = useDispatch();

  return (
    <div>
        <button onClick={() => Dispatch(login("Cathy"))}>Login</button>
    </div>
)
```

What is Unit Testing?

In React, we mainly test:

1. Components (UI behavior)
2. Hooks
3. Utility functions

Unit Testing setup

npm create vite@latest .
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
npm install -D @testing-library/user-event

vitest → Test runner (like Jest but faster)
@testing-library/react → Test React components
@testing-library/jest-dom → Extra matchers (toBeInTheDocument)
jsdom → Simulates browser

setupTests.js inside src folder
import '@testing-library/jest-dom';

package.json and vs.config.js

vs.config.js

```
export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom’,
    setupFiles: './src/setupTests.js’,
  },
})
```

package.json

```
"scripts": {
    "test": "vitest",
 },
```

App.jsx

```
import React from 'react’;  //must for testing
function App() {
  return <h1>Hello World</h1>;
}
export default App;
```

App.test.jsx

```
import React from 'react’;  //must for testing
import { render, screen } from '@testing-library/react';
import App from './App';
test('renders text', () => {
  render(<App />);
  expect(screen.getByText(/hello world/i)).toBeInTheDocument();  //i is case-insensitive
});
```

.....

```
npm run test
```

App1.jsx

```
import React from "react";
import { useState } from "react";
export default function App1() {
  const [message, setMessage] = useState();
  return (
    <div>
      <button onClick={() => setMessage("Hello World")}>Submit</button>
      <p>{message}</p>
    </div>
  );

}
```

App1.test.jsx

```
import React from "react";
import { render,screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import App1 from "./App1";

test("button click", async ()=>{
    render(<App1/>)
    const btn = screen.getByText(/submit/i)
    await userEvent.click(btn)
    const text = screen.getByText("Hello World")
    expect(text).toBeInTheDocument()
})
```

App1.test.jsx

```
import React from "react";
import { render,screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import App1 from "./App1";

test("button click", async ()=>{
    render(<App1/>)
    const btn = screen.getByText(/submit/i)
    await userEvent.click(btn)
    const text = screen.getByText("Hello World")
    expect(text).toBeInTheDocument()
})
```

Login.test.jsx

```
test("Successful Login", async () => {
  render(<Login />);
  const email = screen.getByPlaceholderText("Email");
  const password = screen.getByPlaceholderText("Password");
  const btn = screen.getByText("Login");
  await userEvent.type(email, "admin@test.com");
  await userEvent.type(password, "1234");
  await userEvent.click(btn);
  const text = screen.getByText("Login Successful");
  expect(text).toBeInTheDocument();
});
```

React.memo

Child.jsx

```
import React from "react";
function Child({flag}) {
  console.log("Child called")
  return <div>This is Child Component</div>;
}
export default React.memo(Child);
```

App.jsx

```
    <div>
      <h3>This is App7 Component</h3>
      <button onClick={() => setCounter(counter + 1)}>Update</button>
      <button onClick={() => setFlag(flag + 1)}>Update</button>
      <p>Counter:{counter}</p>
      <p>Flag:{flag}</p>
      <Child flag={flag} />
    </div>
```

useMemo Hook

```
import { useMemo, useState } from "react";
function ExpensiveComponent({ num }) {
  const computedValue = useMemo(() => {
    console.log("Calculating...");
    return num * 2;
  }, [num]);
  return <div>{computedValue}</div>;
}
```

custom Hook - useFetch

....useFetch.js

```
import { useState, useEffect } from "react";
function useFetch(url) {
  const [data, setData] = useState([]);
  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => setData(data));
  }, [url]);
  return data;
}
export default useFetch
```

.....Parent.jsx

```
  const products = useFetch("https://jsonplaceholder.typicode.com/users")
  return <div>{products.length}</div>;
```

useTransition

```
import { useState, useTransition } from "react";
import useFetch from "./useFetch";
function App8() {
  const [text, setText] = useState("");
  const [list, setList] = useState([]);
  const [isPending, startTransition] = useTransition();
  const bigList = useFetch("https://jsonplaceholder.typicode.com/users")
  const handleChange = (e) => {
    const value = e.target.value;
    setText(value);
    startTransition(() => {
      const filtered = bigList.filter(item =>
        item.name.includes(value)
      );
      setList(filtered);
    });
  };
  return (
    <>
      <input onChange={handleChange} />
      {isPending && <p>Loading...</p>}
      {list.map(item => <div key={item.id}>{item.name}</div>)}
    </>
  );
}
export default App8
```

Nested Routes - App

```
<Router>
    <nav>
      <ul><li><Link to="/">Home</Link></li>
        <li><Link to="dashboard">Dashboard</Link></li></ul>
    </nav>
    <Routes>
      <Route path="*" element={<Home />} />
      <Route path="dashboard" element={<Dashboard />}>
        <Route path="profile" element={<Profile />} />
        <Route path="settings" element={<Settings />} />
      </Route>
    </Routes>
  </Router>
```

Nested Routes - Outlet

```
function Dashboard() {
  return (
    <div>
      <h2>Dashboard</h2>
      <nav>
        <ul> <li><Link to="profile">Profile</Link></li>
          <li><Link to="settings">Settings</Link></li></ul>
      </nav>
      <Outlet />
    </div>
  );
}
```

useDeffered hook

```
import { useState, useDeferredValue } from "react";
import useFetch from "./useFetch";
function Search() {
  const [text, setText] = useState("");
  const deferredText = useDeferredValue(text);
  const bigList = useFetch("https://jsonplaceholder.typicode.com/users")
  const filtered = bigList.filter(item =>
    item.includes(deferredText)
  );
  return (
    <>
      <input onChange={(e) => setText(e.target.value)} />
      {filtered.map(item => (
        <div key={item}>{item}</div>
      ))}
    </>
  );
}
```

Layout.jsx

```
function Layout() {
  return (
    <>
      <Header />
      <main>
        <Outlet />
      </main>
      <Footer />
    </>
  );
}
```

App12.jsx – throw new Error

```
export default function App12() {
    throw new Error("App 12 Crashed")
  return (
    <div>App12</div>
  )
}
```

ErrorPage.jsx

```
import { useRouteError } from "react-router-dom";
export default function ErrorPage() {
  const error = useRouteError();
  return (
    <div>
      <h2>Something went wrong</h2>
      <p>{error?.message}</p>
    </div>
  );
}
```

App.jsx - createBrowerRouter

App.jsx

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    errorElement: <GlobalErrorPage />,  //Global Error Handling
    children: [
      {
        index: true,
        element: <Home />
      },
      {
        path: "dashboard",
        element: <Dashboard />,
       errorElement: <ErrorPage />,  //component level Error Handling
      }
    ]
  }
]);
return
<RouterProvider router={router} /
```

Setup Tailwind

npm install -D tailwindcss@3 postcss autoprefixer

npx tailwindcss init –p

Add following in index.css
@tailwind base;
@tailwind components;
@tailwind utilities;

in tailwind.config.js update following

```
content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
```

Basic Tailwind Classes - 1

text-red-500 → text color

bg-blue-500 → background

p-4 → padding - pt-4 , px-4 (left and right), py-3(top and bottom),

m-4 → margin

flex → flexbox

w-1 , w-1/2

```
<h1 class="text-3xl font-bold">Heading</h1>
```

```
<p class="text-sm">Small text</p>
```

Basic Tailwind Classes - 2

```
<div class="border border-gray-300 p-4 rounded-lg">Box</div>

<div class="flex justify-between items-center">
  <span>Left</span>
  <span>Right</span>
</div>

<div class="grid grid-cols-3 gap-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

Responsive Design

```
<div class="text-sm md:text-lg lg:text-2xl">
  Responsive Text
</div>
```

Button Example

```
<button class="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-700">
  Click Me
</button>
```

Hover

```
<div class="bg-gray-200 p-4 hover:bg-gray-400">
  Hover me
</div>
```

Tailwind Reusable Class

```
.box {
  @apply bg-white p-4 rounded shadow;
}
```

Card

```
<div class="max-w-sm mx-auto bg-white shadow-lg rounded-lg p-4">
  <h2 class="text-xl font-bold mb-2">Card Title</h2>
  <p class="text-gray-600">This is a simple card using Tailwind.</p>
  <button class="mt-4 bg-green-500 text-white px-3 py-2 rounded">
    Read More
  </button>
</div>
```

Using Children Props for styling

```
import App19child from './App19child’
export default function App19() {
  return (
    <div>
        <App19child>
            <h1>Hello World</h1>
        </App19child>
    </div>
  )
}
..............................
export default function App19child(props) {
  return (
    <div style={{backgroundColor:'silver’}}>
        {props.children}
    </div>
  )
}
```

Google Authentication

Go to the Google Cloud Console

Create a new project

Navigate to APIs & Services > OAuth consent screen.

Select External > Fill in the app name and support email.

Navigate to Credentials > Create Credentials > OAuth client ID.

Application type: Web application

Authorized JavaScript origins: Add http://localhost:5173

Install Dependencies

npm install @react-oauth/google jwt-decode

1. @react-oauth/google: The main library for the login logic
2. jwt-decode: Used to read the user's details from the token Google returns

Configure the Provider

```
// main.jsx
import React from 'react’
import ReactDOM from 'react-dom/client’
import { GoogleOAuthProvider } from '@react-oauth/google’
import App from './App.jsx’
ReactDOM.createRoot(document.getElementById('root')).render(
   <GoogleOAuthProvider clientId="YOUR_GOOGLE_CLIENT_ID”>
     <App />
   </GoogleOAuthProvider>
)
```

Implement the Login Component

```
// App.jsx
import { GoogleLogin } from '@react-oauth/google’;
import { jwtDecode } from "jwt-decode";
import { useState } from 'react’;
function App() {
  const [user, setUser] = useState(null);
  const handleSuccess = (credentialResponse) => {
    const details = jwtDecode(credentialResponse.credential);
    console.log(details);
    setUser(details);
  };
  return (
    <div>
      {!user ? ( <GoogleLogin   onSuccess={handleSuccess}   onError={() => console.log('Login Failed')} /> ) : (
        <div>
          <img src={user.picture} alt="user" />
          <h3>{user.name}</h3>
          <p>{user.email}</p>
          <button onClick={() => setUser(null)}>Logout</button>
        </div>
      )}
    </div>
  );
}
export default App;
```

What is Next.js?

1. Fast websites. It is built on top of React.
2. SEO-friendly apps
3. Full-stack applications (frontend + backend)

Create Your First Next.js App

npx create-next-app@latest my-appD:\batches\lpu>npx create-next-app@latest lpu-nextjs-app

√ Would you like to use the recommended Next.js defaults? » No, customize settings

√ Would you like to use TypeScript? … No / Yes

√ Which linter would you like to use? » ESLint

√ Would you like to use React Compiler? … No / Yes

√ Would you like to use Tailwind CSS? … No / Yes

√ Would you like your code inside a `src/` directory? … No / Yes

√ Would you like to use App Router? (recommended) … No / Yes

√ Would you like to customize the import alias (`@/*` by default)? … No / Yes

? Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? » No / Yes

Run Your First Next.js App

cd my-app

npm run dev

Open http://localhost:3000

Update app\page.js

```
import Image from "next/image";
export default function Home() {
  return (
    <div className="flex flex-col flex-1 items-center justify-center bg-zinc-50 font-sans dark:bg-
black">
      <h1>Hello World</h1>
    </div>
  );
}
```

.....

Focus on **app/ folder** (modern Next.js)

Routing (File-Based Routing)

1. Create a file:
   - app/about/page.js
     ```
     export default function page() {
       return (
         <div>About Page</div>
       )
     }
     ```
2. Now visit: /about
3. No need for React Router!

Update app\layout.js

```
<body className="min-h-full flex flex-col">
  <h1>Header</h1>
  {children}
  <h4>Footer</h4>
  </body>
```

API Routes (Backend in Next.js)

app/api/products/route.js
Visit: /api/products

```
export async function GET() {
  const products = [
    { id: 1, name: "Product 1", price: 100 },
    { id: 2, name: "Product 2", price: 120 },
    { id: 3, name: "Product 3", price: 140 },
  ];
  return Response.json(products);
}
```

Deployment

Best platform: Vercel

npm run build

Thank You

- PRAVEEN NAIR